

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – II

Roll No.: TY18	Name: Sachin Ramesh Mehta
Class: TYIT	Batch: 1
Date of Assignment: 27/06/2026	Date/Time of Submission: 27/06/2026

PART – A

Write the program with different features of C#

- a. Function Overloading
- b. Inheritance (all types)
- c. Constructor overloading
- d. Interfaces
- e. Using Delegates and events
- f. Exception handling

PART B :

Solution:

Method Overloading :-

Code :-

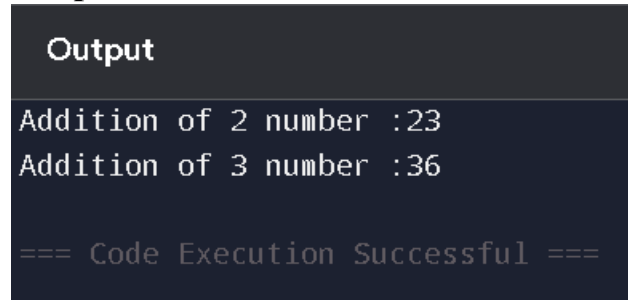
using System;

public class Cal

```
{
    void Num(int num1,int num2)
    {
        Console.WriteLine("Addition of 2 number :"+(num1+num2));
    }
    void Num(int num1,int num2,int num3)
    {
        Console.WriteLine("Addition of 3 number :"+(num1+num2+num3));
    }
    public static void Main(string[] args)
    {
```

```
        Cal obj = new Cal();
        obj.Num(11,12);
        obj.Num(11,12,13);
    }
}
```

Output :-



The screenshot shows a dark-themed window titled "Output". It contains the following text:

```
Addition of 2 number :23
Addition of 3 number :36

=== Code Execution Successful ===
```

Use of constructor

Code :-

```
using System;
```

```
public class Cal
{
    Cal(int num1,int num2)
    {
        Console.WriteLine("Addition of 2 number :"+(num1+num2));
    }
    Cal(int num1,int num2,int num3)
    {
        Console.WriteLine("Addition of 3 number :"+(num1+num2+num3));
    }
    public static void Main(string[] args)
    {

        Cal obj = new Cal(11,12);
        Cal obj1 = new Cal(11,12,13);
    }
}
```

Output :-

Output

```
Addition of 2 number :23
Addition of 3 number :36

=== Code Execution Successful ===
```

Single Inheritance

Code :-

```
using System;
```

```
public class A
{
    public
    void fun1()
    {
        Console.WriteLine("Class A is here !");
    }
}

public class B : A
{
    public
    void fun2(){
        Console.WriteLine("Class B is here !");
    }
}

public class Program
{
    public static void Main(string[] args)
    {
        B obj = new B();
        obj.fun1();
        obj.fun2();
    }
}
```

Output :-

Output

```
Class A is here !  
Class B is here !  
  
=== Code Execution Successful ===
```

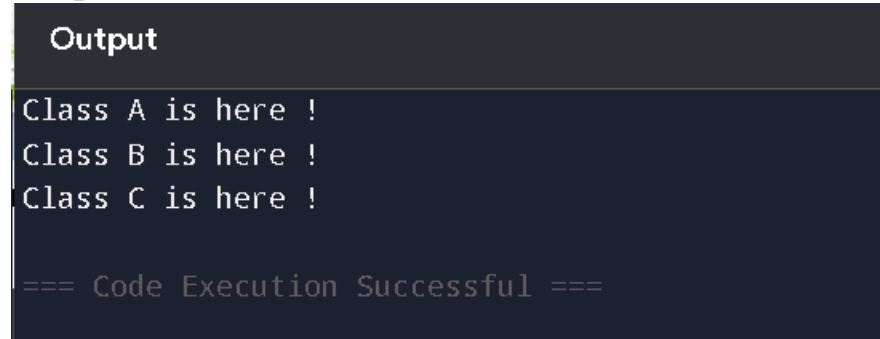
MultiLevel Inheritance

Code :-

```
using System;  
  
public class A  
{  
    public  
    void fun1()  
{  
    Console.WriteLine("Class A is here !");  
}  
}  
  
public class B : A  
{  
    public  
    void fun2(){  
    Console.WriteLine("Class B is here !");  
}  
}  
  
public class C : B  
{  
    public  
    void fun3(){  
    Console.WriteLine("Class C is here !");  
}  
}  
  
public class Program  
{  
    public static void Main(string[] args)  
    {
```

```
        C obj = new C();
        obj.fun1();
        obj.fun2();
        obj.fun3();
    }
}
```

Output :-



The screenshot shows a dark-themed window titled "Output". It contains the following text:

```
Class A is here !
Class B is here !
Class C is here !

=== Code Execution Successful ===
```

Hierarchical Inheritance :-

Code :-

```
using System;

public class A
{
    public
    void fun1()
    {
        Console.WriteLine("Class A is here !");
    }
}

public class B : A
{
    public
    void fun2(){
        Console.WriteLine("Class B is here !");
    }
}

public class C : A
{
    public
    void fun3(){
        Console.WriteLine("Class C is here !");
    }
}
```

```

    }
    }
    public class Program
    {
        public static void Main(string[] args)
        {
            B obj = new B();
            obj.fun1();
            obj.fun2();
            C obj1 = new C();
            obj1.fun1();
            obj1.fun3();
        }
    }
}

```

Output :-

```

    Output
    Class A is here !
    Class B is here !
    Class A is here !
    Class C is here !

    === Code Execution Successful ===

```

Multiple Inheritance :-

Code :-

```

using System;

public interface A
{
    public void fun1();
}
public interface B
{
    public void fun2();
}
public class C : A,B
{

```

```
public void fun1()
{
    Console.WriteLine("Interface A is here !");
}
public void fun2()
{
    Console.WriteLine("Interface B is here !");
}
public void fun3()
{
    Console.WriteLine("Class C is here !");
}
}
}
public class Program
{
    public static void Main(string[] args)
    {

        C obj1 = new C();
        obj1.fun1();
        obj1.fun2();
        obj1.fun3();
    }
}
```

Output :-

Output

```
Interface A is here !
Interface B is here !
Class C is here !
```

```
=== Code Execution Successful ===
```

Using Delegates and events

Code :-

```
using System;

public delegate void Demo();

class Program
{
    public static void Show()
    {
        Console.WriteLine("Hello from Delegate");
    }

    static void Main(string[] args)
    {
        Demo d = Show;
        d();
    }
}
```

Output :-

Output

Hello from Delegate

=== Code Execution Successful ===

Exception Handling

Code :-

```
using System;

class Program
{
    static void Main(string[] args)
    {
        try
        {
            int a = 10;
            int b = 0;
            Console.WriteLine(a / b);
        }

        catch
        {
            Console.WriteLine("Divide by zero error");
        }
    }
}
```

Output :-

Output

Divide by zero error

=== Code Execution Successful ===